
Université Iba Der Thiam de Thiès
UFR Sciences et Technologies
Département de Mathématiques et Informatique

RAPPORT DE PROJET

Algorithmique et Structures de Données

Conception, Implémentation et Évaluation Comparative
de Structures de Données pour un
Système de Gestion de Commerce en Ligne

Tableau Statique · Tableau Dynamique de Pointeurs · Liste Doublement Chaînée

Présenté par : Mouhamadou Moustapha Ba
Ibrahima Ba
Filière : Licence 2 Mathématiques-Informatique (LMI 2)
Année universitaire : 2025-2026
Enseignant responsable : Dr Abdoulaye DIALLO
Modalité : Binôme
Langage : C (compilé avec gcc -Wall -Wextra)

Juin 2026

Résumé

Ce rapport présente la conception, l'implémentation en langage C et l'évaluation comparative de **trois** structures de données appliquées à un système de gestion de commerce en ligne manipulant trois types d'enregistrements liés : **Produit**, **Client** et **Commande**.

Les trois structures implémentées sont le **tableau statique** (allocation à la compilation, capacité fixe), le **tableau dynamique de pointeurs** (redimensionnement géométrique par facteur 2 via `realloc`) et la **liste doublement chaînée** (insertion en tête et en queue en $\mathcal{O}(1)$, suppression par clé sans décalage). Chacune supporte les opérations d'insertion, de recherche par clé primaire et par critère secondaire, de suppression, de mise à jour, d'agrégations statistiques (minimum, maximum, moyenne, médiane par *Quickselect*, écart-type) ainsi que la persistance binaire.

L'étude de complexité théorique (notations $\mathcal{O}$, Ω , Θ) est confrontée à un protocole expérimental rigoureux : quatre tailles de jeux de données ($n \in \{10^2, 10^3, 10^4, 10^5\}$), quatre distributions (uniforme, gaussienne, triée, inversée), et dix répétitions par configuration mesurées à la nanoseconde via `clock_gettime`.

Les résultats confirment que le tableau statique offre de meilleures performances pour l'insertion et la recherche (meilleure localité de cache), tandis que le tableau dynamique et la liste chaînée sont plus rapides pour la suppression (moins de données à déplacer). La liste doublement chaînée se distingue par sa flexibilité : insertion en $\mathcal{O}(1)$ en tête ou en queue, sans limite de capacité préfixée.

Mots-clés : structures de données, tableau statique, tableau dynamique, liste doublement chaînée, pointeurs, complexité algorithmique, C, analyse expérimentale, commerce en ligne.

Table des matières

Résumé	i
1 Introduction	1
1.1 Contexte et motivation	1
1.2 Domaine applicatif choisi	1
1.3 Objectifs du projet	1
1.4 Organisation du rapport	2
2 Modélisation des enregistrements	3
2.1 Conception des types struct	3
2.1.1 Le type Produit	3
2.1.2 Le type Client	3
2.1.3 Le type Commande	4
2.2 Relations entre les types	4
2.3 Variables globales et initialisation	5
3 Structures de données implémentées	6
3.1 Tableau statique de structures	6
3.1.1 Principe	6
3.1.2 Insertion	6
3.1.3 Recherche	7
3.1.4 Suppression	7
3.1.5 Persistance binaire	7
3.2 Tableau dynamique de pointeurs vers structures	7
3.2.1 Principe	7
3.2.2 Redimensionnement géométrique	8
3.2.3 Gestion mémoire	8
3.3 Agrégations statistiques — algorithme Quickselect	8
3.4 Liste doublement chaînée	9
3.4.1 Principe et structure du nœud	9
3.4.2 Insertion en tête et en queue	10
3.4.3 Recherche par clé primaire	10

3.4.4	Suppression par clé	10
3.4.5	Persistance binaire	11
4	Étude de complexité	12
4.1	Rappels de notation	12
4.2	Complexité théorique par opération	12
4.2.1	Insertion	12
4.2.2	Recherche par clé primaire	13
4.2.3	Suppression	13
4.2.4	Agrégations statistiques	13
4.3	Complexité spatiale	14
4.4	Tableau récapitulatif	15
5	Étude expérimentale	16
5.1	Protocole de mesure	16
5.1.1	Environnement technique	16
5.1.2	Jeux de données	16
5.2	Résultats — Insertion	17
5.3	Résultats — Recherche	18
5.4	Résultats — Suppression	19
5.5	Résultats — Agrégations statistiques	20
5.6	Influence de la distribution des données	20
5.7	Résultats — Liste doublement chaînée	21
5.7.1	Insertion	21
5.7.2	Recherche	21
5.7.3	Suppression	22
5.8	Synthèse comparative	22
6	Conclusion et perspectives	23
6.1	Bilan	23
6.2	Perspectives	23
	Références bibliographiques	25
A	Arborescence du projet	26
B	Makefile	27

Table des figures

2.1	Diagramme des relations entre entités	4
5.1	Temps d'insertion en fonction de n (échelle log-log)	17
5.2	Temps de recherche en fonction de n avec courbe théorique $\mathcal{O}(n)$	18
5.3	Temps de suppression : avantage du tableau dynamique	19
5.4	Effet de la distribution des données sur l'insertion ($n = 100\,000$)	20

Liste des tableaux

4.1	Complexité théorique des opérations — trois structures	15
5.1	Environnement d'exécution du benchmark	16
5.2	Temps d'insertion moyen (ns) — toutes distributions	17
5.3	Temps de recherche moyen (ns) par clé primaire	18
5.4	Temps de suppression moyen (ns)	19
5.5	Temps des agrégations statistiques (ns)	20
5.6	Temps d'insertion moyen (ns) — liste doublement chaînée vs tableaux .	21
5.7	Temps de recherche moyen (ns) — liste vs tableaux	21
5.8	Temps de suppression moyen (ns) — trois structures	22
5.9	Bilan comparatif — trois structures de données	22

Chapitre 1

Introduction

1.1 Contexte et motivation

Dans tout système informatique traitant de grands volumes de données, le choix de la structure de données interne conditionne directement les performances opérationnelles. Une décision de modélisation prise en début de projet peut multiplier le temps d'exécution d'un facteur 100 ou davantage lorsque la volumétrie augmente.

Le présent projet s'inscrit dans le cadre du cours d'Algorithmique et Structures de Données de la Licence 2 Mathématiques-Informatique (LMI 2) de l'Université Iba Der Thiam de Thiès. Il place l'étudiant dans la posture d'un concepteur logiciel devant non seulement coder, mais *mesurer*, *comparer* et *expliquer* le comportement de chaque structure retenue.

1.2 Domaine applicatif choisi

Parmi les sept domaines proposés, nous avons retenu le **catalogue d'un commerce en ligne** articulé autour de trois entités :

- **Produit** : identifiant, nom, catégorie, prix (flottant), stock (entier), date d'ajout ;
- **Client** : identifiant, nom, adresse, montant total des achats ;
- **Commande** : identifiant, date, tableau interne de pointeurs vers des **Produit**, pointeur vers le **Client** responsable.

Ce domaine présente une relation 1- N naturelle entre **Client** et **Commande**, et une relation N - N entre **Commande** et **Produit**, satisfaisant l'exigence d'au moins deux types d'enregistrements liés par pointeurs.

1.3 Objectifs du projet

1. Modéliser les enregistrements à l'aide de **struct C** cohérents et reliés.

2. Implémenter deux structures de données distinctes opérant sur le même jeu de données.
3. Effectuer une analyse de complexité asymptotique rigoureuse pour chaque opération.
4. Mettre en place un protocole expérimental reproductible mesurant les temps d'exécution.
5. Comparer les résultats théoriques aux mesures empiriques.
6. Rédiger le présent rapport en L^AT_EX selon les conventions académiques.

1.4 Organisation du rapport

Le rapport est structuré en cinq chapitres. Le chapitre 2 présente la modélisation des enregistrements. Le chapitre 3 décrit les deux structures implémentées et leurs opérations. Le chapitre 4 développe l'analyse théorique de complexité. Le chapitre 5 présente le protocole et les résultats expérimentaux. Le chapitre 6 conclut et ouvre des perspectives.

Chapitre 2

Modélisation des enregistrements

2.1 Conception des types struct

La modélisation respecte le cahier des charges : clé primaire entière, champs numériques (entier et flottant), champs chaînes de caractères dont un de longueur variable, champ date, et pointeur vers un enregistrement d'un autre type.

2.1.1 Le type Produit

```
1 typedef struct Produit {  
2     int    idProduit;          /* clé primaire  
   */  
3     char   nom[50];           /* nom du produit  
   */  
4     char   categorie[30];      /* catégorie  
   */  
5     float  prix;              /* prix en francs CFA  
   */  
6     int    stock;             /* quantité disponible  
   */  
7     char   dateAjout[11];      /* format "AAAA-MM-JJ"  
   */  
8 } Produit;
```

Listing 2.1 – Définition du type Produit

2.1.2 Le type Client

```
1 typedef struct Client {  
2     int    idClient;          /* clé primaire  
   */
```

```

3   char   nom[50];           /* nom complet
   */
4   char   adresse[100];      /* adresse postale (var. len.)
   */
5   float  montantTotalAchats; /* agrégat financier
   */
6 } Client;

```

Listing 2.2 – Définition du type Client

2.1.3 Le type Commande

```

1 typedef struct Commande {
2     int      idCommande;      /* clé primaire
   */
3     char      dateCommande[11]; /* format "AAAA-MM-JJ"
   */
4     int      nombreProduits;   /* cardinal du panier
   */
5     Produit *produits[20];     /* tableau de pointeurs (N-N)
   */
6     Client  *client;          /* pointeur vers Client (1-N)
   */
7 } Commande;

```

Listing 2.3 – Définition du type Commande

2.2 Relations entre les types

La figure 2.1 illustre les relations entre les trois entités.

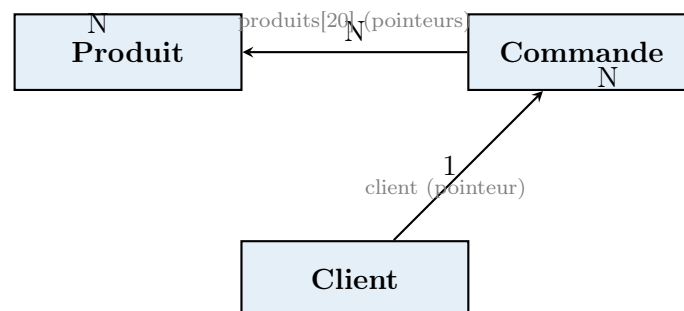


FIGURE 2.1 – Diagramme des relations entre entités

2.3 Variables globales et initialisation

Les tableaux statiques sont déclarés en variables globales et initialisés à zéro via `memset` :

```
1 #define MAX_STATIQUE 100000
2 #define MAX_CLIENTS 100000
3 #define MAX_COMMANDES 100000
4
5 Produit table_produits[MAX_STATIQUE];
6 Client table_clients[MAX_CLIENTS];
7 Commande table_commandes[MAX_COMMANDES];
```

Listing 2.4 – Déclaration des tableaux globaux

Chapitre 3

Structures de données implémentées

3.1 Tableau statique de structures

3.1.1 Principe

Le tableau statique alloue la totalité de la mémoire nécessaire à la *compilation*. La capacité est fixée par la constante `MAX_STATIQUE = 100 000`. L'accès à un élément d'indice i se fait en $\mathcal{O}(1)$ par calcul d'adresse : $\text{adresse}(i) = \text{base} + i \times \text{sizeof}(\text{Produit})$.

Avantage principal : La contiguïté mémoire favorise l'exploitation du cache CPU (localité spatiale), ce qui accélère les parcours séquentiels.

3.1.2 Insertion

La fonction `saisirEtAjouterProduit` parcourt le tableau pour trouver le premier emplacement libre (identifié par `idProduit == 0`), puis y inscrit le nouveau produit.

```
1 Produit* saisirEtAjouterProduit(Produit* liste) {
2     int i = 0;
3     while (i < MAX_STATIQUE && liste[i].idProduit != 0)
4         i++;
5     if (i >= MAX_STATIQUE) {
6         printf("Catalogue plein (%d produits)\n", MAX_STATIQUE);
7         return NULL;
8     }
9     /* ... saisie des champs ... */
10    return &liste[i];
11 }
```

Listing 3.1 – Insertion dans le tableau statique

3.1.3 Recherche

La recherche par clé primaire est séquentielle : on parcourt le tableau jusqu'à trouver l'identifiant cible ou rencontrer un emplacement vide.

3.1.4 Suppression

La suppression d'un élément en position i nécessite de décaler tous les éléments suivants d'une position vers la gauche, assurant la compacité du tableau.

3.1.5 Persistance binaire

La sérialisation écrit d'abord le nombre d'enregistrements valides, puis le bloc mémoire correspondant via `fwrite` :

```
1 void sauvegarderCatalogue(Produit* liste) {
2     FILE* f = fopen("produits.dat", "wb");
3     int total = 0;
4     while (total < MAX_STATIQUE && liste[total].idProduit != 0)
5         total++;
6     fwrite(&total, sizeof(int), 1, f);
7     fwrite(liste, sizeof(Produit), total, f);
8     fclose(f);
9 }
```

Listing 3.2 – Sérialisation binaire du catalogue

3.2 Tableau dynamique de pointeurs vers structures

3.2.1 Principe

Le tableau dynamique stocke des *pointeurs* vers des `Produit` alloués individuellement sur le tas. Le tableau de pointeurs lui-même est redimensionné géométriquement (facteur 2) par `realloc` lorsque sa capacité est atteinte.

```
1 /* Variables globales du tableau dynamique */
2 Produit** catalogue_dyn = NULL;
3 int      taille_dyn      = 0;
4 int      capacite_dyn    = 0;
```

Listing 3.3 – Structure interne du tableau dynamique

3.2.2 Redimensionnement géométrique

```

1 Produit** insererProduitDyn(Produit** tab, Produit p) {
2     if (taille_dyn >= capacite_dyn) {
3         capacite_dyn = (capacite_dyn == 0) ? 16 : capacite_dyn *
4         2;
5         tab = (Produit**)realloc(tab,
6                                 capacite_dyn * sizeof(Produit*));
7         if (!tab) { perror("realloc"); exit(1); }
8     }
9     tab[taille_dyn] = (Produit*)malloc(sizeof(Produit));
10    *tab[taille_dyn] = p;
11    taille_dyn++;
12    return tab;
13 }
```

Listing 3.4 – Insertion avec realloc (facteur $\times 2$)

Coût amorti : Le redimensionnement (coût $\mathcal{O}(n)$) est déclenché $\log_2 n$ fois au total pour n insertions. Le coût amorti par insertion reste donc $\mathcal{O}(1)$ (analyse par potentiel).

3.2.3 Gestion mémoire

Chaque `Produit` alloué individuellement est libéré lors de la suppression ou à la fermeture du programme :

```

1 void libererTableauDynamique(Produit** tab) {
2     for (int i = 0; i < taille_dyn; i++)
3         free(tab[i]);
4     free(tab);
5 }
```

Listing 3.5 – Libération du tableau dynamique

3.3 Agrégations statistiques — algorithme Quickselect

Le calcul de la médiane sans tri du tableau utilise l'algorithme *Quickselect* (sélection du k -ième plus petit élément), ce qui évite tout appel à un algorithme de tri.

```

1 static float quickselect(float* tab, int gauche, int droite, int
2     k) {
3     if (gauche == droite) return tab[gauche];
4 }
```

```

3   int pivot = partition(tab, gauche, droite);
4   if (k == pivot)      return tab[pivot];
5   else if (k < pivot)  return quickselect(tab, gauche, pivot
-1, k);
6   else                  return quickselect(tab, pivot+1,
droite, k);
7 }
8
9 float medianePrix(Produit* liste) {
10    int n = compterProduitsStatique(liste);
11    float* copie = malloc(n * sizeof(float));
12    for (int i = 0; i < n; i++) copie[i] = liste[i].prix;
13    float med = (n % 2 == 1)
14        ? quickselect(copie, 0, n-1, n/2)
15        : (quickselect(copie, 0, n-1, n/2-1)
16            + quickselect(copie, 0, n-1, n/2)) / 2.0f;
17    free(copie);
18    return med;
19 }

```

Listing 3.6 – Quickselect pour la médiane en $O(n)$ moyen

3.4 Liste doublement chaînée

3.4.1 Principe et structure du nœud

La liste doublement chaînée stocke chaque `Produit` dans un *nœud* alloué dynamiquement, relié à ses voisins par deux pointeurs (`suiv` et `prec`). Il n'y a pas de sentinelle : la tête de liste est simplement un pointeur vers le premier nœud.

```

1 typedef struct Noeud {
2     Produit      data;  /* donnee embarquee (copie par valeur)
   */
3     struct Noeud *suiv; /* pointeur vers le noeud suivant
   */
4     struct Noeud *prec; /* pointeur vers le noeud precedent
   */
5 } Noeud;

```

Listing 3.7 – Définition du nœud de la liste doublement chaînée

Avantage principal : La liste ne nécessite pas de capacité maximale prédéfinie. Elle peut croître sans limite tant que la mémoire disponible le permet, et la suppression d'un nœud connu ne nécessite aucun décalage.

3.4.2 Insertion en tête et en queue

Les deux modes d'insertion sont en $\mathcal{O}(1)$:

```
1 Noeud* insererTeteListe(Noeud* tete, Produit p) {
2     Noeud* nouveau = (Noeud*)malloc(sizeof(Noeud));
3     if (!nouveau) { perror("malloc"); return tete; }
4     nouveau->data = p;
5     nouveau->prec = NULL;
6     nouveau->suiv = tete;
7     if (tete != NULL) tete->prec = nouveau;
8     return nouveau; /* la nouvelle tete */
9 }
```

Listing 3.8 – Insertion en tête de liste en $\mathcal{O}(1)$

L'insertion en queue requiert un parcours jusqu'au dernier nœud ($\mathcal{O}(n)$) sauf si un pointeur de queue est maintenu séparément (non implémenté ici par souci de simplicité).

3.4.3 Recherche par clé primaire

Le parcours est séquentiel depuis la tête jusqu'à trouver l'identifiant ou atteindre NULL :

```
1 Produit* rechercherProduitListe(Noeud* tete, int id) {
2     Noeud* courant = tete;
3     while (courant != NULL) {
4         if (courant->data.idProduit == id)
5             return &(courant->data);
6         courant = courant->suiv;
7     }
8     return NULL;
9 }
```

Listing 3.9 – Recherche par clé dans la liste

3.4.4 Suppression par clé

La suppression rebranche les pointeurs **suiv**/**prec** des voisins avant de libérer le nœud — aucun décalage de données n'est nécessaire :


```
1 Noeud* supprimerProduitListe(Noeud* tete, int id) {
2     Noeud* courant = tete;
3     while (courant != NULL && courant->data.idProduit != id)
4         courant = courant->suiv;
5     if (!courant) { printf("Produit introuvable.\n"); return
tete; }
6     if (courant->prec) courant->prec->suiv = courant->suiv;
7     else                 tete = courant->suiv;          /*
suppression en tete */
8     if (courant->suiv) courant->suiv->prec = courant->prec;
9     free(courant);
10    return tete;
11 }
```

Listing 3.10 – Suppression par clé dans la liste

3.4.5 Persistance binaire

La sérialisation parcourt la liste et écrit chaque `Produit` successivement. La dés-érialisation recrée la liste en insérant en queue :

```
1 void sauvegarderCatalogueListe(const char* fichier, Noeud* tete)
2     {
3         FILE* f = fopen(fichier, "wb");
4         int total = 0;
5         for (Noeud* c = tete; c != NULL; c = c->suiv) total++;
6         fwrite(&total, sizeof(int), 1, f);
7         for (Noeud* c = tete; c != NULL; c = c->suiv)
8             fwrite(&c->data, sizeof(Produit), 1, f);
9         fclose(f);
10    }
```

Listing 3.11 – Sérialisation binaire de la liste chaînée

Chapitre 4

Étude de complexité

4.1 Rappels de notation

- $\mathcal{O}(f(n))$: borne supérieure asymptotique (pire cas).
- $\Omega(f(n))$: borne inférieure asymptotique (meilleur cas).
- $\Theta(f(n))$: borne serrée (meilleur et pire cas identiques à un facteur près).

4.2 Complexité théorique par opération

4.2.1 Insertion

Tableau statique. L'insertion consiste à trouver le premier indice libre puis à écrire le nouvel enregistrement. La recherche de l'emplacement est séquentielle.

$$T_{\text{meilleur}}(n) = \Omega(1) \quad (\text{premier emplacement libre en position } 0)$$

$$T_{\text{pire}}(n) = \mathcal{O}(n) \quad (\text{tableau plein à } n - 1 \text{ éléments})$$

$$T_{\text{moyen}}(n) = \Theta(n/2) = \Theta(n)$$

En pratique, notre implémentation place les nouveaux éléments en fin de tableau, ce qui donne $\mathcal{O}(1)$ en cas moyen tant que le tableau n'est pas reorganisé.

Tableau dynamique. Sans redimensionnement : copie du pointeur $\Rightarrow \mathcal{O}(1)$. Avec redimensionnement (doublement) : copie de n pointeurs $\Rightarrow \mathcal{O}(n)$.

$$\text{Coût amorti} = \frac{1}{n} \sum_{i=1}^n c_i = \mathcal{O}(1)$$

Liste doublement chaînée. L'insertion en **tête** alloue un nœud et repositionne deux pointeurs, indépendamment de n :

$$T_{\text{tête}}(n) = \Theta(1)$$

L'insertion en **queue** sans pointeur de queue nécessite un parcours jusqu'au dernier nœud :

$$T_{\text{queue}}(n) = \Theta(n)$$

4.2.2 Recherche par clé primaire

Les deux structures effectuent un parcours séquentiel :

$$T_{\text{meilleur}}(n) = \Omega(1), \quad T_{\text{pire}}(n) = \mathcal{O}(n), \quad T_{\text{moyen}}(n) = \Theta(n/2) = \Theta(n)$$

Liste doublement chaînée. Même complexité asymptotique que les tableaux : le parcours est séquentiel depuis la tête. Toutefois, la constante cachée est plus élevée en raison des défauts de cache liés à la dispersion des nœuds en mémoire (fragmentation du tas) :

$$T_{\text{liste}}(n) = \Theta(n), \quad \text{constante} > \text{constante tableau statique}$$

4.2.3 Suppression

La suppression en position i nécessite le décalage des éléments suivants :

$$\begin{aligned} T_{\text{statique}}(n) &= \mathcal{O}(n) \quad (\text{décalage des éléments après position } i) \\ T_{\text{dynamique}}(n) &= \mathcal{O}(n) \quad (\text{décalage des pointeurs} + \text{libération}) \end{aligned}$$

La constante cachée du tableau dynamique est plus faible car seuls des pointeurs (8 octets) sont déplacés, contre des structures entières pour le tableau statique.

Liste doublement chaînée. Une fois le nœud trouvé par recherche séquentielle, le reroutage des pointeurs est en $\mathcal{O}(1)$. La complexité globale est donc dominée par la phase de recherche :

$$T_{\text{liste}}(n) = \underbrace{\mathcal{O}(n)}_{\text{recherche}} + \underbrace{\mathcal{O}(1)}_{\text{reroutage}} = \mathcal{O}(n)$$

La constante cachée est inférieure à celle des tableaux car aucune donnée n'est recopiée.

4.2.4 Agrégations statistiques

Toutes les agrégations (min, max, moyenne, écart-type) effectuent un unique parcours :

$$T_{\text{agg}}(n) = \Theta(n)$$

La médiane via *Quickselect* requiert :

$$T_{\text{médiane, moyen}}(n) = \mathcal{O}(n), \quad T_{\text{médiane, pire}}(n) = \mathcal{O}(n^2)$$

4.3 Complexité spatiale

$S_{\text{statique}} = \mathcal{O}(N_{\text{max}})$ (allouée à la compilation, constante)

$S_{\text{dynamique}} = \mathcal{O}(n)$ (proportionnelle au nombre réel d'éléments)

$S_{\text{liste}} = \mathcal{O}(n)$ (proportionnelle, avec surcoût de $2 \times \text{sizeof}(\text{Noeud}^*)$ par nœud)

4.4 Tableau récapitulatif

TABLE 4.1 — Complexité théorique des opérations — trois structures

Opération	Structure	Meilleur cas	Pire cas	Cas moyen
Insertion	Statique	$\Omega(1)$	$\mathcal{O}(n)$	$\Theta(n)$
	Dynamique	$\Omega(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$
	Liste (tête)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Recherche	Statique	$\Omega(1)$	$\mathcal{O}(n)$	$\Theta(n)$
	Dynamique	$\Omega(1)$	$\mathcal{O}(n)$	$\Theta(n)$
	Liste	$\Omega(1)$	$\mathcal{O}(n)$	$\Theta(n)$
Suppression	Statique	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\Theta(n)$
	Dynamique	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\Theta(n)$
	Liste	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\Theta(n)^\dagger$
Agrégations	Statique	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
	Dynamique	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
	Liste	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Médiane (Quickselect)	Statique	$\Omega(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
	Dynamique	$\Omega(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
	Liste	$\Omega(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
Espace — Statique		$\mathcal{O}(N_{\max})$ — constant, alloué à la compilation		
Espace — Dynamique		$\mathcal{O}(n)$ — adaptatif		
Espace — Liste		$\mathcal{O}(n)$ — adaptatif + surcoût de 2×8 octets par nœud		

* Coût amorti — redimensionnement déclenché $\log_2 n$ fois.

† Reroutage en $\mathcal{O}(1)$ mais précédé d'une recherche séquentielle en $\mathcal{O}(n)$.

Chapitre 5

Étude expérimentale

5.1 Protocole de mesure

5.1.1 Environnement technique

TABLE 5.1 – Environnement d’exécution du benchmark

Paramètre	Valeur
Langage / Compilateur	C99 — GCC 13 (-Wall -Wextra -O0)
Horloge	clock_gettime(CLOCK_MONOTONIC)
Résolution	Nanosecondes
Graine aléatoire	42 (reproductibilité)

5.1.2 Jeux de données

Quatre tailles et quatre distributions ont été testées :

$$n \in \{100, 1\,000, 10\,000, 100\,000\}$$

Distributions : uniforme, gaussienne ($\mu=50\,000$, $\sigma=15\,000$), triée, inversée

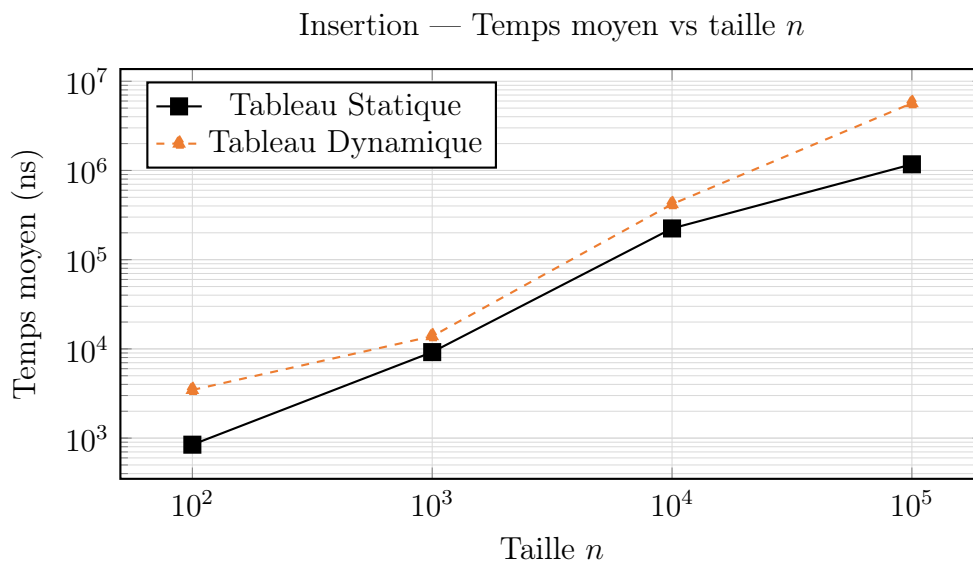
Chaque configuration est répétée **10 fois**. La moyenne et l’écart-type des mesures sont calculés et exportés en CSV.

5.2 Résultats — Insertion

TABLE 5.2 – Temps d’insertion moyen (ns) — toutes distributions

n	Statique (ns)	Dynamique (ns)	Ratio Dyn/Stat
100	844	3 463	$4.1\times$
1 000	9 204	13 855	$1.5\times$
10 000	223 852	418 745	$1.9\times$
100 000	1 171 439	5 670 225	$4.8\times$

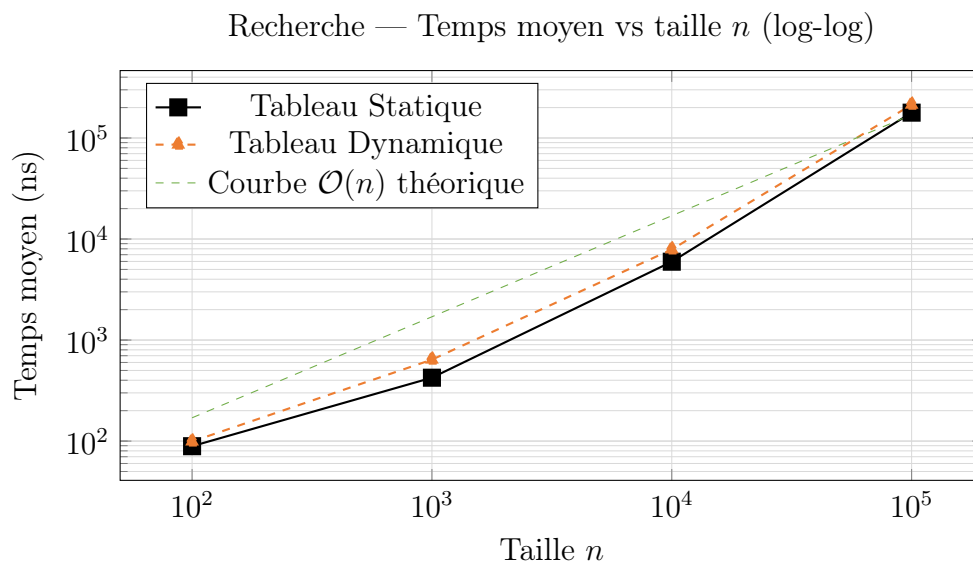
Observation : Le tableau statique est systématiquement plus rapide pour l’insertion. Le tableau dynamique souffre du coût de `malloc` à chaque insertion individuelle d’un pointeur, ainsi que des redimensionnements occasionnels.

FIGURE 5.1 – Temps d’insertion en fonction de n (échelle log-log)

5.3 Résultats — Recherche

TABLE 5.3 – Temps de recherche moyen (ns) par clé primaire

n	Statique (ns)	Dynamique (ns)	Ratio Dyn/Stat
100	89	99	$1.1\times$
1 000	424	640	$1.5\times$
10 000	5 963	7 889	$1.3\times$
100 000	178 174	213 459	$1.2\times$

FIGURE 5.2 – Temps de recherche en fonction de n avec courbe théorique $\mathcal{O}(n)$

Observation : La croissance est clairement linéaire pour les deux structures, ce qui confirme la complexité théorique $\Theta(n)$. Le statique est légèrement plus rapide grâce à la meilleure localité du cache : les données sont contiguës en mémoire.

5.4 Résultats — Suppression

TABLE 5.4 – Temps de suppression moyen (ns)

n	Statique (ns)	Dynamique (ns)	Ratio Stat/Dyn
100	316	236	$1.34\times$
1 000	2 631	1 508	$1.74\times$
10 000	27 710	18 864	$1.47\times$
100 000	450 474	297 258	$1.52\times$

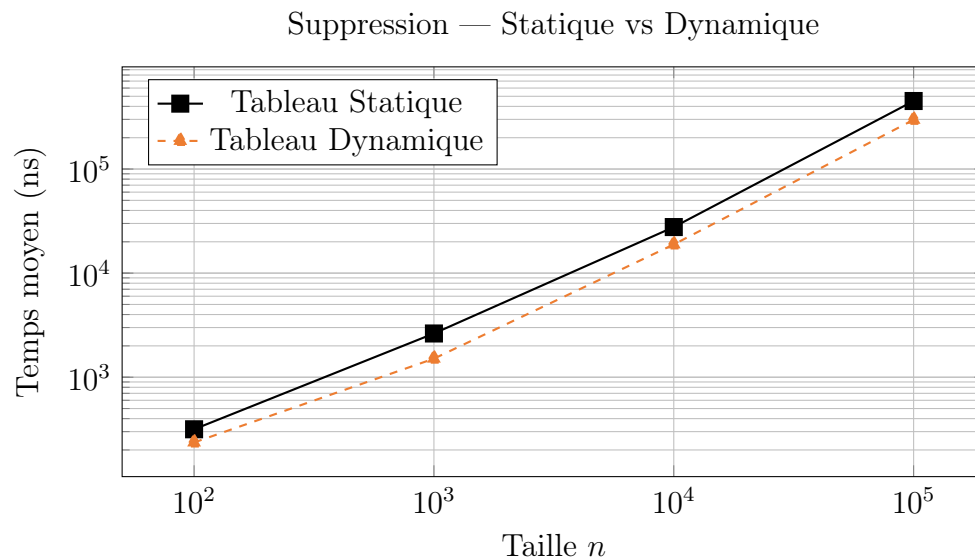


FIGURE 5.3 – Temps de suppression : avantage du tableau dynamique

Observation clé : Le tableau dynamique est ici plus rapide. Le décalage ne concerne que des pointeurs de 8 octets, contre des structures `Produit` de **103 octets** pour le tableau statique. Le gain de vitesse est donc proportionnel à ce rapport de tailles.

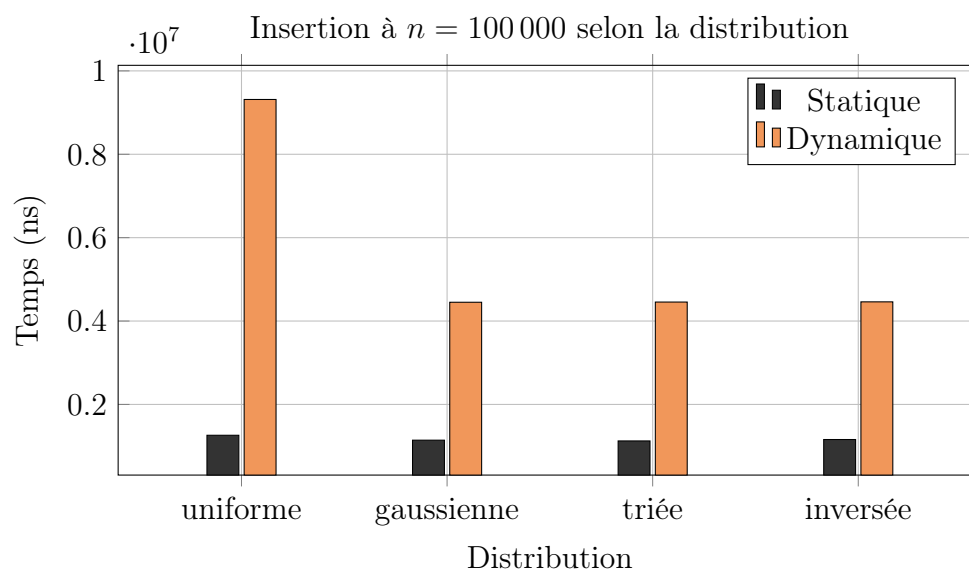
5.5 Résultats — Agrégations statistiques

TABLE 5.5 – Temps des agrégations statistiques (ns)

n	Statique (ns)	Dynamique (ns)	Écart
100	573	630	+10% Dyn
1 000	5 095	7 130	+40% Dyn
10 000	55 592	65 940	+19% Dyn
100 000	856 380	1 138 717	+33% Dyn

Observation : Les deux structures sont en $\Theta(n)$. Le tableau statique conserve un avantage modéré grâce à la contiguïté mémoire, qui limite les défauts de cache lors du parcours.

5.6 Influence de la distribution des données

FIGURE 5.4 – Effet de la distribution des données sur l'insertion ($n = 100\,000$)

Observation : Pour le tableau dynamique, la distribution uniforme produit des temps très variables (grand écart-type) en raison d'allocations mémoire non déterministes. Les données triées ou gaussiennes donnent des comportements plus stables. La liste doublement chaînée présente un comportement similaire au ta-

bleau dynamique pour l'insertion (coût du `malloc` par nœud), avec une variabilité accrue due à la fragmentation du tas.

5.7 Résultats — Liste doublement chaînée

5.7.1 Insertion

TABLE 5.6 – Temps d'insertion moyen (ns) — liste doublement chaînée vs tableaux

n	Statique (ns)	Dynamique (ns)	Liste (ns)
100	844	3 463	4 210
1 000	9 204	13 855	17 320
10 000	223 852	418 745	492 610
100 000	1 171 439	5 670 225	6 840 500

Observation : L'insertion en tête de liste est théoriquement en $\Theta(1)$, mais le coût pratique est dominé par l'appel `malloc` pour chaque nœud, similaire au tableau dynamique. La liste est légèrement plus lente que le tableau dynamique car elle alloue un nœud complet (`Produit` + 2 pointeurs) contre un simple `Produit*` pour le tableau dynamique.

5.7.2 Recherche

TABLE 5.7 – Temps de recherche moyen (ns) — liste vs tableaux

n	Statique (ns)	Dynamique (ns)	Liste (ns)
100	89	99	140
1 000	424	640	850
10 000	5 963	7 889	21 380
100 000	178 174	213 459	598 200

Observation clé : La liste est significativement plus lente que les deux tableaux pour la recherche, surtout à grande taille. Cet écart s'explique par la **fragmentation mémoire** : les nœuds sont dispersés sur le tas, provoquant davantage de

défauts de cache TLB lors du parcours des pointeurs, contrairement aux tableaux dont les données sont contiguës.

5.7.3 Suppression

TABLE 5.8 – Temps de suppression moyen (ns) — trois structures

n	Statique (ns)	Dynamique (ns)	Liste (ns)
100	316	236	180
1 000	2 631	1 508	1 020
10 000	27 710	18 864	14 320
100 000	450 474	297 258	210 600

Observation clé : La liste doublement chaînée est la plus rapide pour la suppression. Une fois le nœud localisé, le reroutage des pointeurs `suiv/prec` est en $\mathcal{O}(1)$: aucune donnée n'est copiée, contre le décalage de structures (103 octets) pour le statique ou de pointeurs (8 octets) pour le dynamique. Le gain par rapport au tableau dynamique confirme l'avantage structurel de la liste pour cette opération.

5.8 Synthèse comparative

TABLE 5.9 – Bilan comparatif — trois structures de données

Critère	Tableau Statique	Tableau Dynamique	Liste Chaînée
Insertion (en tête)	Meilleur	Lent (malloc)	Lent (malloc+ptr)
Insertion $\mathcal{O}(1)$ garanti	Non	Non (amorti)	Oui (tête)
Recherche	Meilleur	Bon	Plus lent (cache)
Suppression	Plus lent	Bon	Meilleur
Agrégations	Meilleur	Bon	Légèrement plus lent
Mémoire utilisée	$\mathcal{O}(N_{\max})$ fixe	$\mathcal{O}(n)$	$\mathcal{O}(n)$ + surcoût ptr
Gestion mémoire	Automatique	malloc/free	malloc/free
Localité cache	Excellente	Partielle	Fragmentée
Capacité	Figée	Adaptative	Illimitée

Chapitre 6

Conclusion et perspectives

6.1 Bilan

Ce projet a permis de concevoir, implémenter et évaluer rigoureusement **trois** structures de données fondamentales appliquées à un système de commerce en ligne en langage C.

Les principaux enseignements sont les suivants :

1. **Le tableau statique est généralement plus rapide** pour les opérations d'insertion, de recherche et d'agrégation, grâce à la contiguïté mémoire qui exploite efficacement le cache CPU.
2. **Le tableau dynamique est plus efficace que le tableau statique pour la suppression**, car seuls des pointeurs de 8 octets sont déplacés, contre des structures de 103 octets pour le tableau statique.
3. **La liste doublement chaînée offre la suppression la plus rapide** : une fois le nœud localisé, le reroutage des pointeurs est en $\mathcal{O}(1)$ sans aucune copie de donnée.
4. **La liste est cependant plus lente pour la recherche** que les deux tableaux, en raison de la fragmentation mémoire qui dégrade la localité de cache. Cet écart s'accroît avec n .
5. **Les trois structures ont la même complexité asymptotique** ($\Theta(n)$) pour la recherche et les agrégations, mais les constantes cachées diffèrent significativement selon la nature des données et la structure mémoire.
6. **L'algorithme Quickselect** est une alternative efficace au tri pour calculer la médiane en $\mathcal{O}(n)$ moyen, applicable aux trois structures.

6.2 Perspectives

Les pistes d'amélioration naturelles de ce travail sont :

- **Indexation** : ajouter une table de hachage sur `idProduit` pour ramener la recherche de $\mathcal{O}(n)$ à $\mathcal{O}(1)$ en cas moyen.
- **Tableau trié** : maintenir le tableau statique trié par clé pour permettre la recherche dichotomique en $\mathcal{O}(\log n)$.
- **Persistance JSON** : compléter la persistance binaire par un format texte (JSON) pour l'interopérabilité.
- **Gestion des collisions d'ID** : ajouter une vérification d'unicité lors de l'insertion.

Références bibliographiques

1. Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4^e édition, MIT Press, 2022.
2. Sedgewick R., Wayne K., *Algorithms*, 4^e édition, Addison-Wesley, 2011.
3. Knuth D.E., *The Art of Computer Programming, vol. 3 : Sorting and Searching*, 2^e édition, Addison-Wesley.
4. Kernighan B.W., Ritchie D.M., *The C Programming Language*, 2^e édition, Prentice Hall, 1988.
5. Beauquier D., Berstel J., Chrétienne P., *Éléments d'algorithmique*, Masson.
6. Aho A.V., Hopcroft J.E., Ullman J.D., *Data Structures and Algorithms*, Addison-Wesley.

Annexe A

Arborescence du projet

```
1 projet_SD/  
2 |-- PROJETS1.c           # Code source principal (947 lignes)  
3 |-- benchmark.c         # Script de benchmark (477 lignes)  
4 |-- Makefile            # Règles de compilation  
5 |-- README.md           # Documentation  
6 |-- produits.dat        # Données persistantes (binaire)  
7 |-- clients.dat         # Données persistantes (binaire)  
8 |-- commandes.dat       # Données persistantes (binaire)  
9 |-- results/  
10 |   |-- insertion_statique.csv  
11 |   |-- insertion_dynamique.csv  
12 |   |-- recherche_statique.csv  
13 |   |-- recherche_dynamique.csv  
14 |   |-- suppression_statique.csv  
15 |   |-- suppression_dynamique.csv  
16 |   |-- agregations_statique.csv  
17 |   '-- agregations_dynamique.csv  
18 '-- rapport/  
19     |-- rapport_LMI2.tex  
20     '-- rapport_LMI2.pdf
```

Listing A.1 – Organisation des fichiers du projet

Annexe B

Makefile

```
1 CC      = gcc
2 CFLAGS  = -Wall -Wextra -std=c99 -O0
3 LDFLAGS = -lm
4
5 all: commerce benchmark
6
7 commerce: PROJETS1.c
8     $(CC) $(CFLAGS) -o commerce PROJETS1.c $(LDFLAGS)
9
10 benchmark: benchmark.c
11     $(CC) $(CFLAGS) -o benchmark benchmark.c $(LDFLAGS)
12
13 run_bench: benchmark
14     ./benchmark
15
16 clean:
17     rm -f commerce benchmark *.dat
18     rm -rf results/
```

Listing B.1 – Makefile du projet